فصل ۷: اعتبارسنجی متقابل (cross-validation) و گردش کار (workflow)

مقدمه

این شاید مهمترین فصل باشد. سوال اساسی مطرح شده در این فصل به شرح زیر است:
چگونه مدلی را انتخاب کنیم که خوب پیش‌بینی کند؟

این هدف از اعتبارسنجی متقابل است، صرف نظر از اینکه مدل چیست. این روش کمی متفاوت با آمارهای سنتی است که بیشتر به چگونگی درک بهتر یک پدیده می‌پردازند. یادگیری ماشین مربوط به پیش‌بینی و چگونگی پردازش الگوریتم یادگیری ماشین در داده‌های دیده نشده جدید و رسیدن به پیش‌بینی‌ها است. حتی اگرشبیه آمار سنتی نباشد، می‌توانید از تفسیر و درک دامنه برای ایجاد ستون‌های جدید (ویژگی ها) استفاده کنید و حتی پیش‌بینی‌های بهتری را انجام دهید. برای ایجاد ستون‌های جدید می‌توانید از آمارهای سنتی استفاده کنید.

در اوایل کتاب، ما با تقسیم آموزش/ آزمایش داده‌ها شروع کردیم. "اعتبارسنجی متقابل" تکرار بسیاری از تقسیمات مهم داده‌های آموزشی و آزمایش برای به حداکثر رساندن عملکرد پیش‌بینی است. این فصل موارد زیر را بررسی می‌کند.

- طرح‌های اعتبارسنجی متقابل
- بهترین پارامترهای یک تخمین زننده کدامند؟

• معیارهایی که y_test را با y_pred مقایسه می‌کنند- هدف واقعی در مقایسه با مجموعه هدف پیش‌بینی شده

انتخاب مدلی با اعتبارسنجی

عملکرد cross_val_score را در فصول قبل مشاهده کردیم. کاری که اینجا انجام می‌دهیم بسیار شبیه آن خواهد بود، با این تفاوت که ما از دو ستون آخر مجموعه داده iris استفاده خواهیم کرد. هدف از این بخش انتخاب بهترین مدل است. قبل از شروع، بهترین مدل را مدلی تعریف می‌کنیم که بالاترین امتیاز را داشته باشد. اگر امتیازها برابر بود، ما مدلی را انتخاب می‌کنیم که بهترین امتیاز را با کمترین نوسان داشته باشد.

در این دستورالعمل موارد زیر را انجام خواهیم داد:

- دو ویژگی آخر (ستون) مجموعه داده iris را بارگیری کنید
- داده‌ها را به داده‌های آموزش و آزمایش تقسیم کنید
- دو الگوریتم k-نزدیک‌ترین همسایگان (KNN) را با سه و پنج همسایه ایجاد کنید
- امتیاز دهی دو الگوریتم
- مدلی را انتخاب کنید که بهترین امتیاز را داشته باشد

با بارگذاری مجموعه داده شروع کنید

```
from sklearn import datasets
iris = datasets.load_iris()
X = iris.data[:,2:]
y = iris.target
```

داده‌ها را به آموزش و آزمایش تقسیم کنید.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify =y,random_state = 7)
```

ابتدا دو الگوریتم نزدیک‌ترین همسایه را نمونه‌سازی کنید

```
from sklearn.neighbors import KNeighborsClassifier
kn_3 = KNeighborsClassifier(n_neighbors = 3)
kn_5 = KNeighborsClassifier(n_neighbors = 5)
```

اکنون، هر دو الگوریتم را با استفاده از cross_val_score امتیاز دهید. در kn_3_scores و kn_5_scores لیستی از امتیازات را مشاهده کنید

```
from sklearn.model_selection import cross_val_score
kn_3_scores = cross_val_score(kn_3, X_train, y_train, cv=4)
kn_5_scores = cross_val_score(kn_5, X_train, y_train, cv=4)
print(kn_3_scores)
output: array([0.96428571, 0.92857143, 0.89285714, 1.    ])
print(kn_5_scores)
output: array([0.96428571, 0.96428571, 0.89285714, 1.    ])
```

مشاهده میانگین و انحراف معیار دو مدل

```
print ("Mean of kn_3: ",kn_3_scores.mean())
print ("Mean of kn_5: ",kn_5_scores.mean())
output: Mean of kn_3: 0.9464285714285714
Mean of kn_5: 0.9553571428571429
print ("Std of kn_3: ",kn_3_scores.std())
print ("Std of kn_5: ",kn_5_scores.std())
output:
Std of kn_3: 0.039929785312496226
Std of kn_5: 0.03891874056732743
```

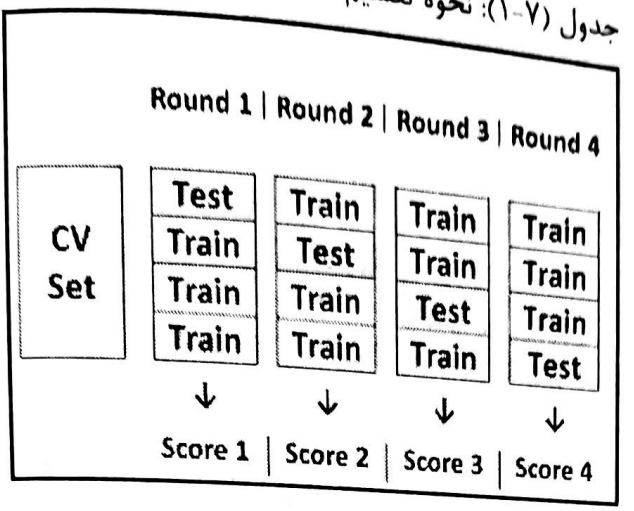
به طور کلی، kn_5، هنگامی که الگوریتم روی پنج همسایه تنظیم می‌شود، عملکرد کمی بهتر از سه همسایه است، اما از ثبات کمتری برخوردار است (امتیازات آن در کل کمی بیشتر است).

اکنون مرحله آخر را انجام می‌دهیم: مدلی را انتخاب کنید که بالاترین امتیاز را داشته باشد. بنابراین kn_5 را انتخاب می‌کنیم زیرا بالاترین امتیاز را دارد. (توجه داشته باشید که امتیازات در نظر گرفته شده نزدیک‌ترین همسایگان دارای امتیاز دقت پیش فرض هستند: نسبت طبقه‌بندی‌های صحیح تقسیم بر تمام طبقه بندی‌های انجام شده.)

روش کار k-fold

این یک نمونه از اعتبارسنجی ۴ بخش است زیرا در عملکرد cross_val_score، cv = 4. ما داده‌های آموزش یا X_train را به چهار قسمت یا بخش تقسیم می‌کنیم. ما با چرخش هر بخش به عنوان مجموعه آزمایش این فرآیند را تکرار می‌کنیم. در ابتدا، بخش ۱ مجموعه تست است در حالی که بخش ۲، ۳ و ۴ با هم مجموعه آموزشی هستند. سپس بخش ۲ مجموعه تست است در حالی که ۱، ۳ و ۴ مجموعه آموزشی است. این روش را با بخش‌های ۳ و ۴ نیز تکرار می‌کنیم.

جدول (۷-۱): نحوه تقسیم داده‌ها در اعتبارسنجی متقابل



هنگامی که مجموعه داده را به بخش‌هایی تقسیم کردیم، به الگوریتم چهار بار امتیاز می‌دهیم

۱. ما یکی از الگوریتم‌های نزدیک‌ترین همسایگان را روی بخش‌های ۲، ۳ و ۴ آموزش می‌دهیم.

۲. آزمایش را روی بخش ۱ انجام می‌دهیم.

۳. حال دقت طبقه‌بندی را اندازه‌گیری می‌کنیم: بخش آزمایش را با نتایج پیش‌بینی شده در آن بخش مقایسه کنید؛ و این اولین نمره طبقه‌بندی در لیست است.

این فرآیند چهار بار انجام می‌شود. خروجی نهایی لیستی از چهار نمره است.

به طور کلی، ما کل فرآیند را دو بار انجام دادیم، یک بار برای kn_3 و یک بار برای kn_5، و دو لیست برای انتخاب بهترین مدل تولید کردیم. (ماژول استفاده شده model_selection است).

K-fold cross validation

یک مجموعه داده بسازید که بسیار کوچک باشد سپس KFold را وارد کرده و تعداد بخش‌بندی‌ها (fold) را انتخاب کنید

```
import numpy as np
X = np.array([[1, 2], [3, 4], [5, 6], [7, 8],[1, 2], [3, 4], [5, 6], [7,8]])
y = np.array([1, 2, 1, 2, 1, 2, 1, 2])
from sklearn.model_selection import KFold
kf= KFold(n_splits = 4)
cc = 1
cc = 1
for train_index, test_index in kf.split(X):
```

```
print ("Round ",cc,": ","Training indices :", train_index,"Testing indices :",
test_index)
cc += 1
output
Round  1 : Training indices : [2 3 4 5 6 7] Testing indices : [0 1]
Round  2 : Training indices : [0 1 4 5 6 7] Testing indices : [2 3]
Round  3 : Training indices : [0 1 2 3 6 7] Testing indices : [4 5]
Round  4 : Training indices : [0 1 2 3 4 5] Testing indices : [6 7]Testing indices
: [6 7]
```

همچنین می‌توانید تعداد بخش‌ها را مشاهده کنیم

```
kf.get_n_splits()
output: 4
```

اگر بخواهید، می‌توانید داده‌ها را در خود بخش‌ها مشاهده کنید.

```
indices_list = list(kf.split(X))
```

اکنون، indices_list لیستی از tuples است و می‌توانید اطلاعات مربوط به چهارمین بخش را مشاهده کنید

```
indices_list[3] #the list is indexed from 0 to 3
output: (array([0, 1, 2, 3, 4, 5]), array([6, 7]))
```

این اطلاعات با اطلاعات قبلی مطابقت دارد، با این تفاوت که به صورت مجموعه‌ای از دو آرایه NumPy است. داده‌های واقعی را از چهارمین بخش مشاهده کنید. داده‌های آموزش را برای چهارمین بخش مشاهده کنید

```
train_indices, test_indices = indices_list[3]
X[train_indices]
Output: array([[1, 2],   [3, 4],   [5, 6],   [7, 8],   [1, 2],   [3, 4]])
y[train_indices]
output: array([1, 2, 1, 2, 1, 2])
X[test_indices]
Output: array([[5, 6],   [7, 8]])
y[test_indices]
output: array([1, 2])
```

اعتبارسنجی متعادل

در حالی که مجموعه داده‌ها را به چندین بخش مختلف تقسیم می‌کنید، ممکن است از خود بپرسید: آیا مجموعه‌های مختلف در هر اعتبارسنجی متقابل k-fold نمی‌تواند بسیار متفاوت باشد؟ توزیع‌ها می‌توانند در هر بخش بسیار متفاوت باشند و این اختلافات می‌تواند

به نوسان امتیازات منجر شود. با کمک از اعتبارسنجی، یک راه‌حل وجود دارد. زیرمجموعه‌های مجموعه داده همانند نسخه‌های کوچک‌تر از کل مجموعه داده (حداقل در متغیر هدف) به نظر می‌رسند.

اگر اعتبارسنجی 4-fold را روی این مجموعه داده انجام دهیم، هر چهار تا از آزمایشات فقط یک مقدار برای هدف دارند. این کار با استفاده از StratifiedKFold قابل اصلاح است.

```
import numpy as np
X = np.array([[1, 2], [3, 4], [5, 6], [7, 8],[1, 2], [3, 4], [5, 6], [7,8]])
y = np.array([1, 1, 1, 1, 2, 2, 2, 2])
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits = 4)
cc = 1
for train_index, test_index in skf.split(X,y):
 print ("Round",cc,":","Training indices :", train_index," Testing indices :",
test_index)
 cc += 1
```

```
output:
Round 1 : Training indices : [1 2 3 5 6 7] Testing indices : [0 4]
Round 2 : Training indices : [0 2 3 4 6 7] Testing indices : [1 5]
Round 3 : Training indices : [0 1 3 4 5 7] Testing indices : [2 6]
Round 4 : Training indices : [0 1 2 4 5 6] Testing indices : [3 7]
```

مشاهده کنید که روش تقسیم کلاس skf، تقسیم طبقه‌بندی k بخش، دارای دو آرگومان X و y است که تلاش می‌کند تا مقدار هدف y را با توزیع مشابه در هر یک از مجموعه‌های برابر توزیع کند. در این حالت، هر زیر مجموعه ۵۰٪ داده‌ها را دارند.

می‌توانید از StratifiedShuffleSplit استفاده کنید تا بخش‌های طبقه‌بندی شده را دوباره اصلاح کنید. توجه داشته باشید که با استفاده از مجموعه‌های آزمایشی که جدا از یکدیگر نیستند، چهار بخش ایجاد نمی‌شود.

```
from sklearn.model_selection import StratifiedShuffleSplit
sss = StratifiedShuffleSplit(n_splits = 5,test_size=0.25)
cc = 1
for train_index, test_index in sss.split(X,y):
 print ("Round",cc,":", "Training indices :", train_index, "Testing indices :",
test_index)
 cc += 1
```

```
output
Round 1 : Training indices : [2 3 1 5 6 4] Testing indices : [0 7]
Round 2 : Training indices : [7 5 0 4 1 2] Testing indices : [3 6]
Round 3 : Training indices : [1 4 5 2 0 7] Testing indices : [6 3]
Round 4 : Training indices : [5 4 3 0 7 2] Testing indices : [1 6]
Round 5 : Training indices : [3 4 5 0 6 2] Testing indices : [1 7]
```

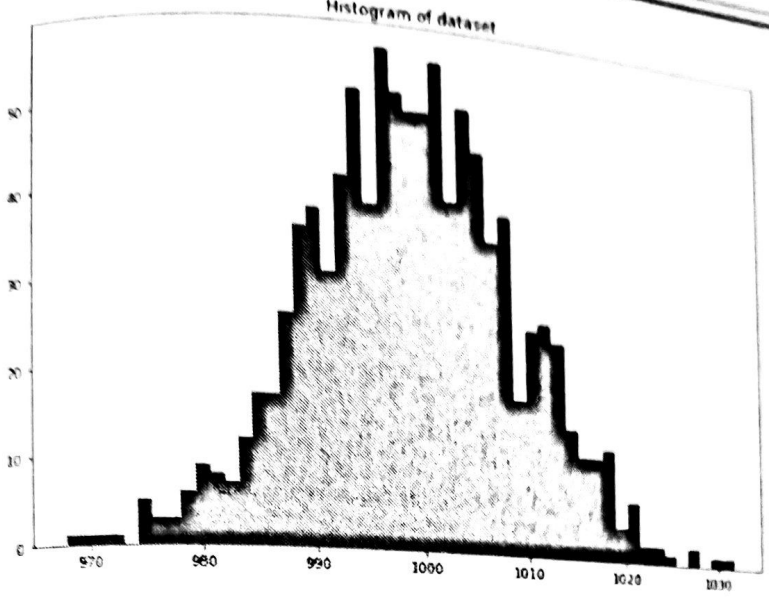
این تقسیم‌ها، تقسیم مجموعه داده نیستند بلکه تکرار یک رویه تصادفی هستند، هر کدام با اندازه مجموعه آموزش ۷۵٪ از کل مجموعه داده و اندازه مجموعه آزمایش ۲۵٪ است.

اعتبارسنجی متقابل با ShuffleSplit

ShuffleSplit یکی از ساده‌ترین تکنیک‌های اعتبارسنجی متقابل است. با استفاده از این روش اعتبارسنجی متقابل، به سادگی نمونه‌ای از داده‌ها برای تعداد تکرارهای مشخص شده تهیه می‌شود. ما مثالی از تخمین میانگین یک مجموعه داده تک متغیره را مرور خواهیم کرد. این شبیه نمونه‌گیری مجدد است، اما نشان خواهد داد که چرا ما می‌خواهیم از اعتبارسنجی متقابل استفاده کنیم.

ابتدا باید مجموعه داده‌ها را ایجاد کنیم. ما از NumPy برای ایجاد یک مجموعه داده استفاده خواهیم کرد که در آن میانگین اصلی را می‌دانیم. ما نیمی از مجموعه داده را تخمین می‌زنیم تا ببینیم چقدر به میانگین اصلی نزدیک است. یک نمونه تصادفی توزیع شده معمول با میانگین برابر ۱۰۰۰ و انحراف معیار برابر ۱۰ ایجاد می‌کنیم

```
%matplotlib inline
import numpy as np
true_mean = 1000
true_std = 10
N = 1000
dataset = np.random.normal(loc= true_mean, scale = true_std,
size=N)
import matplotlib.pyplot as plt
f, ax = plt.subplots(figsize=(10, 7))
ax.hist(dataset, color='k', alpha=.65,
histtype='stepfilled',bins=50)
ax.set_title("Histogram of dataset")
```



شکل (۷-۱): داده‌ها با میانگین ۱۰۰۰ و انحراف معیار ۱۰

میانگین نصفی از داده‌ها

```
holdout_set = dataset[:500]
fitting_set = dataset[500:]
estimate = fitting_set[:N//2].mean()
print(estimate)
```

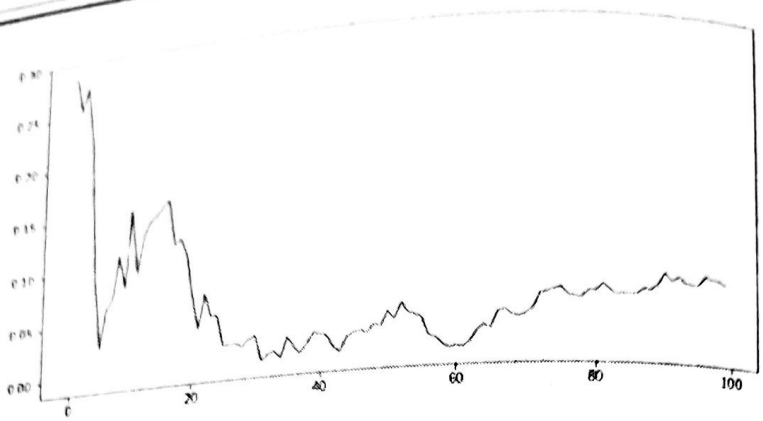
```
999.5549933429049
```

میانگین کل داده‌ها

```
data_mean = dataset.mean()
print(data_mean)
```

```
999.5635991979591
```

```
from sklearn.model_selection import ShuffleSplit
shuffle_split = ShuffleSplit(n_splits=100, test_size=.5,random_state=0)
mean_p = []
estimate_closeness = []
for train_index, not_used_index in  shuffle_split.split(fitting_set):
 mean_p.append(fitting_set[train_index].mean())
 shuf_estimate = np.mean(mean_p)
 estimate_closeness.append(np.abs(shuf_estimate -dataset.mean()))
plt.figure(figsize=(10,5))
plt.plot(estimate_closeness)
```



شکل (۷-۲): نمایش تخمین بر اساس تعداد مراکز خوشه ها

میانگین تخمین زده شده به میانگین داده‌ها ۹۹۹/۵۶۳۵۹۹۱۹۷۹۵۹۱ نزدیک‌تر می‌شود و تنها حدود ۰/۰۰۸۶۰۵۸۵۵۰۵۴۱۵۶۵۶۴ با هم اختلاف دارند.

اعتبارسنجی سری زمانی

scikit-learn می‌تواند اعتبارسنجی متقابل را برای داده‌های سری زمانی مانند داده‌های بازار سهام انجام دهد. ما این کار را با تقسیم سری زمانی انجام خواهیم داد، زیرا می‌خواهیم مدل آینده را پیش‌بینی کند.

ما شاخص‌ها را برای تقسیم سری زمانی ایجاد خواهیم کرد. با ایجاد یک مجموعه داده کوچک شروع کنید.

```
from sklearn.model_selection import TimeSeriesSplit
import numpy as np
X = np.array([[1, 2], [3, 4], [1, 2], [3, 4],[1, 2], [3, 4], [1, 2], [3,4]])
y = np.array([1, 2, 3, 4, 1, 2, 3, 4])
tscv = TimeSeriesSplit(n_splits=7)
for train_index, test_index in tscv.split(X):
  X_train, X_test = X[train_index], X[test_index]
  y_train, y_test = y[train_index], y[test_index]
  print ("Training indices:", train_index, "Testing indices:",  test_index)

tscv_list = list(tscv.split(X))
output:
Training indices: [0] Testing indices: [1]
Training indices: [0 1] Testing indices: [2]
Training indices: [0 1 2] Testing indices: [3]
Training indices: [0 1 2 3] Testing indices: [4]
```

```
Training indices: [0 1 2 3 4] Testing indices: [5]
Training indices: [0 1 2 3 4 5] Testing indices: [6]
Training indices: [0 1 2 3 4 5 6] Testing indices: [7]
```

همچنین می‌توانید پنجره‌های متحرک را با NumPy یا پانداها ایجاد کنید. نیاز اصلی اعتبارسنجی سری زمانی این است که مجموعه آزمون پس از تنظیم آموزش در زمان مشخص ظاهر شود. در غیر این صورت، شما می‌توانید گذشته را از آینده پیش‌بینی کنید. اعتبارسنجی سری زمانی جالب توجه است زیرا بسته به مجموعه داده، تأثیر زمان متفاوت است. بعضی اوقات، نیازی نیست که ردیف‌های داده را به ترتیب زمانی قرار دهید، اما هرگز نمی‌توانید تصور کنید که آینده را در گذشته می‌دانید.

جستجوی شبکه (Grid-searching) با scikit-learn

در ابتدای فصل انتخاب مدل و اعتبارسنجی متقابل، سعی کردیم بهترین مدل نزدیک‌ترین همسایه را برای دو ویژگی آخر مجموعه داده iris انتخاب کنیم. اکنون با GridSearchCV این کار را انجام خواهیم داد.

نکته:

جستجوی شبکه یا Grid-searching فرآیند اسکن داده‌ها برای پیکربندی پارامترهای بهینه برای یک مدل داده شده است؛ و بسته به نوع مدل مورد استفاده، پارامترهای خاصی لازم است.

```
from sklearn import datasets
iris = datasets.load_iris()
X = iris.data[:,2:]
y = iris.target
from sklearn.model_selection import train_test_split,cross_val_score
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify= y,random_state
= 7)
from sklearn.neighbors import KNeighborsClassifier
knn_clf = KNeighborsClassifier()
```

یک شبکه پارامتر تهیه کنید، که برای جستجوی شبکه لازم است. شبکه پارامترها یک dictionary با تنظیم پارامتری است که می‌خواهید امتحان کنید

```
param_grid = {'n_neighbors': list(range(3,9,1))}
```

جستجوی شبکه را با ارسال موارد زیر به عنوان آرگومان انجام دهید:

- تخمین زننده
- شبکه پارامتر
- نوعی اعتبارسنجی متقابل، cv = 10

```
from sklearn.model_selection import GridSearchCV
gs = GridSearchCV(knn_clf,param_grid,cv=10)
gs.fit(X_train, y_train)
```

نتایج را می‌توان با دستور زیر مشاهده نمود:

```
print(gs.best_params_)
output: {'n_neighbors': 3}

gs.cv_results_['mean_test_score']
print(list(zip(gs.cv_results_['params'],gs.cv_results_['mean_test_score'])))
output:
[({'n_neighbors': 3}, 0.9553030303030303), ({'n_neighbors': 4},
0.9378787878787879), ({'n_neighbors': 5}, 0.9553030303030303),
({'n_neighbors': 6}, 0.9469696969696969), ({'n_neighbors': 7},
0.9553030303030303), ({'n_neighbors': 8}, 0.9553030303030303)]
all_scores = []
for n_neighbors in range(3,9,1):
  knn_clf = KNeighborsClassifier(n_neighbors = n_neighbors)
  all_scores.append((n_neighbors, cross_val_score(knn_clf, X_train,y_train,
cv=10).mean()))
  sorted(all_scores, key = lambda x:x[1], reverse = True)
print(all_scores)
output: [(3, 0.9553030303030303), (4, 0.9378787878787879), (5,
0.9553030303030303), (6, 0.9469696969696969), (7, 0.9553030303030303),
(8, 0.9553030303030303)]
```

مشکل این روش این است که زمانبرتر و مستعد خطا است، به خصوص اگر تعداد پارامترها بیشتر یا تغییرات اضافی ایجاد شوند، مانند مواردی که در استفاده از خطوط لوله دخیل هستند. توجه داشته باشید که جستجوی شبکه و brute force هر دو مقادیر ممکن پارامترها را اسکن می‌کند.

جستجوی تصادفی

از نقطه نظر عملی، RandomizedSearchCV مهم‌تر از جستجوی منظم شبکه است. این بدان دلیل است که با مقدار متوسطی از داده‌ها یا با مدلی که شامل چند پارامتر است،

محاسبه ترکیب هر پارامتری در شبکه کامل بسیار هزینه‌بر است. منابع محاسباتی بهتر است برای طبقه‌بندی نمونه برداری یا بهبود رویه‌های تصادفی صرف شوند.

```
from sklearn import datasets
iris = datasets.load_iris()
X = iris.data[:,2:]
y = iris.target
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify = y,random_state = 7)
```

طبقه‌بندی نزدیک‌ترین همسایه را انجام دهید:

```
from sklearn.neighbors import KNeighborsClassifier
knn_clf = KNeighborsClassifier()
```

توزیع پارامتر را که برای جستجوی تصادفی در شبکه لازم است، آماده کنید. توزیع پارامتر دیکشنری توسط تنظیم پارامتری است که می‌خواهید به طور تصادفی امتحان کنید

```
param_dist = {'n_neighbors': list(range(3,9,1))}
```

یک شی جستجوی تصادفی در شبکه ایجاد و موارد زیر را به عنوان آرگومان ارسال کنید:

- تخمین زننده
- توزیع پارامتر
- نوعی اعتبارسنجی متقابل، cv = 10
- تعداد دفعات اجرای روش، n_iter

```
from sklearn.model_selection import RandomizedSearchCV
rs = RandomizedSearchCV(knn_clf,param_dist,cv=10,n_iter=6)
rs.fit(X_train, y_train)
print("Best parameters ",rs.best_params_)
print(list(zip(rs.cv_results_['params'],rs.cv_results_['mean_test_score'])))
```

output:

```
Best parameters {'n_neighbors': 12}

[({'n_neighbors': 30}, 0.9462121212121211), ({'n_neighbors': 19},
0.9378787878787879), ({'n_neighbors': 25}, 0.9462121212121211),
({'n_neighbors': 12}, 0.9643939393939395), ({'n_neighbors': 11},
0.9643939393939395), ({'n_neighbors': 21}, 0.9462121212121211),
({'n_neighbors': 34}, 0.9462121212121211), ({'n_neighbors': 31},
0.9371212121212121), ({'n_neighbors': 44}, 0.9462121212121211),
({'n_neighbors': 41}, 0.9462121212121211), ({'n_neighbors': 18},
```

0.9462121212121211), ({'n_neighbors': 46}, 0.9462121212121211),
({'n_neighbors': 43}, 0.9462121212121211), ({'n_neighbors': 4},
0.9378787878787879), ({'n_neighbors': 7}, 0.9553030303030303)]

در این حالت، ما در واقع جستجوی شبکه را از طریق شش پارامتر انجام دادیم. با این وجود می‌توانستید فضای پارامتر بزرگ‌تری را اسکن کنید

```
param_dist = {'n_neighbors': list(range(3,50,1))}
rs = RandomizedSearchCV(knn_clf,param_dist,cv=10,n_iter=15)
rs.fit(X_train,y_train)
print(rs.best_params_)
output : {'n_neighbors': 14}
```

زمان اجرا را می‌توان مشاهده کرد

```
%timeit rs.fit(X_train,y_train)
526 ms ± 22.4 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
from sklearn.model_selection import GridSearchCV
param_grid = {'n_neighbors': list(range(3,50,1))}
gs = GridSearchCV(knn_clf,param_grid,cv=10)
%timeit gs.fit(X_train,y_train)
Output:1.61 s ± 47.7 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
gs.best_params_
Output:{'n_neighbors': 11}
print(list(zip(gs.cv_results_['params'],gs.cv_results_['mean_test_score'])))
Output: [({'n_neighbors': 3}, 0.9553030303030303), ({'n_neighbors': 4},
0.9378787878787879), ({'n_neighbors': 5}, 0.9553030303030303),
({'n_neighbors': 6}, 0.9469696969696969), ({'n_neighbors': 7},
0.9553030303030303), ({'n_neighbors': 8}, 0.9553030303030303),
({'n_neighbors': 9}, 0.9553030303030303), ({'n_neighbors': 10},
0.9462121212121211), ({'n_neighbors': 11}, .....
```

نکته:

اینکه از جستجوی تصادفی استفاده کنید یا خیر تصمیمی است که باید به صورت موردی بگیرید. برای اینکه بتوانید یک الگوریتم را حس کنید باید از یک جستجوی تصادفی استفاده نمایید. بدون در نظر گرفتن پارامترها، ممکن است عملکرد ضعیفی داشته باشد، بنابراین می‌توانید از الگوریتم دیگری استفاده کنید. اگر الگوریتم عملکرد بسیار خوبی داشته باشد، می‌توانید با جستجوی شبکه‌ای کامل بهترین پارامترها را پیدا کنید. علاوه بر این، به جای تمرکز بر جستجوهای جامع، می‌توانید مجموعه‌ای از الگوریتم‌ها را ترکیب کنید.

معیارهای طبقه‌بندی

در اوایل فصل، انتخاب بهترین نمونه از نزدیک‌ترین همسایگان را بر اساس تعداد همسایگان و پارامتر n_neighbours، بررسی کردیم. این پارامتر اصلی در طبقه‌بندی نزدیک‌ترین همسایگان است: یک نقطه را بر اساس برچسب KNN طبقه‌بندی کنید. بنابراین، برای ۳ نزدیک‌ترین همسایه، یک نقطه را بر اساس برچسب سه نقطه نزدیک طبقه‌بندی کنید. سه نقطه‌ای را انتخاب کنید که اکثریت آرا را داشته باشند.

معیار accuracy_score، معیار طبقه‌بندی بود که به عنوان تعداد طبقه بندی‌ها است. معیارهای دیگری نیز وجود دارد و ما در اینجا آن‌ها را بررسی خواهیم کرد.

```
import pandas as pd
data_web_address ="pima-indians-diabetes.csv"
column_names = ['pregnancy_x',
'plasma_con',
'blood_pressure',
'skin_mm',
'insulin',
'bmi',
'pedigree_func', 'age','target']
feature_names = column_names[:-1]
all_data = pd.read_csv(data_web_address , names=column_names)
```

داده‌ها را به مجموعه‌های آموزشی و آزمایشی تقسیم کنید

```
import numpy as np
import pandas as pd
X = all_data[feature_names]
y = all_data['target']
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=0.2,
random_state=7,stratify=y)
```

برای مرور بخش قبلی، یک جستجوی تصادفی را با استفاده از الگوریتم KNN انجام دهید

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import RandomizedSearchCV
knn_clf = KNeighborsClassifier()
param_dist = {'n_neighbors': list(range(3,20,1))}
rs = RandomizedSearchCV(knn_clf,param_dist,cv=10,n_iter=17)
rs.fit(X_train, y_train)
```

نمایش بهترین دقت

```
rs.best_score_
output: 0.7539397144368059
```

علاوه بر این، به ماتریس سردرگمی مجموعه تست نگاه کنید

```
y_pred = rs.predict(X_test)
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, y_pred)
output:
array([[84, 16],
  [27, 27]], dtype=int64)
```

ماتریس سردرگمی اطلاعات ویژه‌تری در مورد نحوه عملکرد مدل ارائه می‌دهد. ۲۷ نفر که دیابت نداشتند به اشتباه جز بیماران تشخیص داده شدند . این اشتباه جدی‌تر از ۱۶ نفری است که تصور می‌کردند به دیابت مبتلا هستند که واقعاً چنین نبود.

در این شرایط، ما می‌خواهیم حساسیت را به حداکثر برسانیم یا یادآوری (recall) کنیم.

هنگام بررسی مدل‌های خطی، ما به تعریف یادآوری (recall) یا حساسیت پرداختیم.

$$\text{Sensitivity} = \frac{People\ correctly\ labelled\ having\ diabets}{All\ People\ who\ have\ diabets}$$

$$\text{Sensitivity} = TPR = \frac{27}{27+27} = 0.5$$

حساسیت مجموعه را با استفاده از y_test و y_pred اندازه بگیرید

```
from sklearn.metrics import recall_score
recall_score(y_test, y_pred)
output: 0.5
from sklearn.metrics import make_scorer
recall_scorer = make_scorer(recall_score, greater_is_better=True)
```

اکنون یک جستجوی تصادفی در شبکه انجام دهید

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import RandomizedSearchCV
knn_clf = KNeighborsClassifier()
param_dist = {'n_neighbors': list(range(3,20,1))}
rs =
RandomizedSearchCV(knn_clf,param_dist,cv=10,n_iter=17,scoring=recall_scorer)
rs.fit(X_train, y_train)
```

اکنون به بالاترین امتیاز نگاه کنید

```
rs.best_score_
output: 0.5647186147186147
```

به نمره فراخوان نگاه کنید

```
y_pred = rs.predict(X_test)
recall_score(y_test,y_pred)
output: 0.5
```

در جستجوی تصادفی می‌توانستید نمره roc_auc_sc، منطقه ROC زیر منحنی را امتحان کنید

```
from sklearn.metrics import roc_auc_score
rs
=RandomizedSearchCV(knn_clf,param_dist,cv=10,n_iter=17,scoring=make_sc
orer(roc_auc_score,greater_is_better=True))
rs.fit(X_train, y_train)
rs.best_score_
output: 0.7098593073593072
```

امتیازدهنده سفارشی

شما می‌توانید امتیاز دهنده خود را برای طبقه‌بندی طراحی کنید. فرض کنید شما یک شرکت بیمه هستید و هزینه‌های مربوط به هر سلول در ماتریس سردرگمی را با خود داشته‌اید. هزینه‌های نسبی به شرح زیر است

جدول (۷-۲): امتیازدهی سفارشی

شخص نادرست برچسب‌گذاری شده و شرایط دارد $2 برای تست اضافی	شخص به درستی برچسب‌گذاری شده که هیچ شرایطی ندارد $1
شخص درست برچسب‌گذاری شده و شرایط دارد $4	شخص به درستی برچسب‌گذاری نشده که هیچ شرایطی ندارد $100

هزینه ماتریس سردرگمی به شرح زیر محاسبه می‌شود

```
costs_array = confusion_matrix(y_test, y_pred) * np.array([[1,2],[100,20]])
costs_array
```

اکنون آن را در یک امتیاز دهنده قرار دهید و جستجوی شبکه را اجرا کنید. آرگومان درون امتیاز دهنده greater_is_betteris روی False تنظیم شده است، زیرا هزینه‌ها باید تا حد ممکن کم باشد

```
def costs_total(y_test, y_pred):
 return (confusion_matrix(y_test, y_pred) * np.array([[1,2], [100,20]])).sum()
costs_scorer = make_scorer(costs_total, greater_is_better=False)
rs
=RandomizedSearchCV(knn_clf,param_dist,cv=10,n_iter=17,scoring=costs_sc
orer)
rs.fit(X_train, y_train)
rs.best_score_
```

این نمره منفی است زیرا وقتی آرگومان greater_is_better در make_scorerfunction باشد، امتیاز در ۱- ضرب می‌شود. جستجوی شبکه سعی در به حداکثر رساندن این امتیاز دارد، در نتیجه مقدار مطلق امتیاز را به حداقل می‌رساند. هزینه مجموعه تست به شرح زیر است:

```
costs_total(y_test,rs.predict(X_test))
```

در حالی که به این تعداد نگاه می‌کنید، فراموش نکنید که به تعداد افراد شرکت کننده در مجموعه آزمون که ۱۵۴ نفر است، توجه کنید. هزینه متوسط هر نفر حدود ۲۱/۸ دلار است.

معیارهای رگرسیون

اعتبارسنجی متقابل با یک متغیر رگرسیون در scikit امکان‌پذیر است. تابع امتیازدهی sklearn.metrics وارد کنید و آن را در یک تابع make_scorer قرار دهید، یا می‌توانید یک امتیازدهنده سفارشی ایجاد کنید.

مجموعه داده‌ای را که از معیار رگرسیون استفاده می‌کند بارگیری کنید. ما مجموعه داده‌های مسکن بوستون را بارگیری می‌کنیم و آن را به مجموعه‌های آموزشی و آزمایشی تقسیم می‌کنیم

```
from sklearn.datasets import load_boston
boston = load_boston()
X = boston.data
y = boston.target
from sklearn.model_selection import train_test_split, cross_val_score
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2,random_state=7)
```

ما اطلاعات زیادی در مورد مجموعه داده نداریم. می‌توانیم با استفاده از الگوریتم واریانس بالا یک جستجوی سریع در شبکه را امتحان کنیم

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import RandomizedSearchCV
```

```
knn_reg = KNeighborsRegressor()
param_dist = {'n_neighbors': list(range(3,20,1))}
rs = RandomizedSearchCV(knn_reg,param_dist,cv=10,n_iter=17)
rs.fit(X_train, y_train)
rs.best_score_
output: 0.46457900083535553
from sklearn.linear_model import Ridge
cross_val_score(Ridge(),X_train,y_train,cv=10).mean()
output: 0.7443218509488487
```

هر دو تنظیم کننده، به طور پیش فرض، r2_score، R-squared را محاسبه می‌کنند.
بنابراین مدل خطی به مراتب بهتر است. یک مدل پیچیده متفاوت، مجموعه‌ای از درختان را امتحان کنید

```
from sklearn.ensemble import
GradientBoostingRegressor,RandomForestRegressor
cross_val_score(GradientBoostingRegressor(max_depth=7),X_train,y_train,cv=
10).mean()
output: 0.8279554005259749
```

عملکرد این گروه حتی بهتر است. همچنین می‌توانید یک جنگل تصادفی را امتحان کنید:

```
cross_val_score(RandomForestRegressor(),X_train,y_train,cv=10).mean()
output: 0.8541674209648347
```

اکنون می‌توانیم با به حداکثر رساندن R-squared بر روی ساخت gradient boosting تمرکز می‌کنیم. یک یا دو جستجوی تصادفی را امتحان کنید.

```
param_dist = {'n_estimators': [4000], 'learning_rate':
[0.01],'max_depth':[1,2,3,5,7]}
rs_inst_a = RandomizedSearchCV(GradientBoostingRegressor(),
param_dist,n_iter = 5, n_jobs=1)
rs_inst_a.fit(X_train, y_train)
```

بهینه‌سازی برای R-squared موارد زیر را برمی گرداند

```
rs_inst_a.best_params_
output:{'n_estimators': 4000, 'max_depth': 3, 'learning_rate': 0.01}
```

همان طور که قابل مشاهده است برای درخت عمق سه انتخاب شده است.

```
rs_inst_a.best_score_
output:0.8888752445706108
```

اکنون موارد زیر را انجام خواهیم داد:

۱. یک تابع امتیازدهی ایجاد کنید.
۲. با آن تابع scikit-scorer بسازید.

۳. برای یافتن بهترین پارامترهای gradient boost و برای به حداقل رساندن تابع خطا، روی جستجوی شبکه را اجرا کنید.

```
def mape_score(y_test, y_pred):
 return (np.abs(y_test - y_pred)/y_test).mean()
from sklearn.metrics import make_scorer
mape_scorer = make_scorer(mape_score, greater_is_better=False)
param_dist = {'n_estimators': [4000], 'learning_rate':
[0.01],'max_depth':[1,2,3,4,5]}
rs_inst_b = RandomizedSearchCV(GradientBoostingRegressor(),param_dist,
n_iter = 3, n_jobs=1,scoring = mape_scorer)
rs_inst_b.fit(X_train, y_train)
rs_inst_b.best_score_
output: -0.1060534356448101
rs_inst_b.best_params_
output: {'n_estimators': 4000, 'max_depth': 4, 'learning_rate': 0.01}
```

در این مثال عمق درخت ۴ انتخاب شد

معیارهای خوشه‌بندی

اندازه‌گیری عملکرد یک الگوریتم خوشه‌بندی کمی پیچیده‌تر از طبقه‌بندی یا رگرسیون است، زیرا خوشه‌بندی یادگیری ماشین بدون نظارت است. خوشبختانه، scikit-learn به ما در این امر نیز بسیار کمک می‌کند. امتیازدهی به الگوریتم خوشه‌بندی بسیار شبیه به الگوریتم طبقه‌بندی است.

برای اندازه‌گیری عملکرد خوشه‌بندی، از مجموعه داده iris استفاده می‌کنیم. گل‌های iris را به دو نوع برچسب‌گذاری می‌کنیم: نوع صفر هر زمان که هدف صفر باشد و نوع ۱ وقتی که هدف ۱ یا ۲ باشد:

```
from sklearn.datasets import load_iris
import numpy as np
iris = load_iris()
X = iris.data
y = np.where(iris.target == 0,0,1)
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=2,random_state=0)
kmeans.fit(X)
from sklearn.metrics.cluster import adjusted_rand_score
from sklearn.metrics import make_scorer
from sklearn.model_selection import cross_val_score
```

```
cross_val_score(kmeans,X,y,cv=10,scoring=make_scorer(adjusted_rand_score)
).mean()
output: 0.8733695652173914
```

استفاده از تخمین زننده ساختگی برای مقایسه نتایج

این دستورالعمل در مورد ایجاد تخمین زننده است. داشتن یک نقطه مرجع برای مدلی که در نهایت خواهید ساخت ارزشمند است.

در این دستورالعمل، کارهای زیر را انجام خواهیم داد:

۱. چند داده تصادفی ایجاد کنید.

۲. برازش کردن با تخمین زننده‌های مختلف ساختگی.

این دو مرحله را برای داده‌های رگرسیون و داده‌های طبقه‌بندی انجام خواهیم داد

ابتدا داده‌های تصادفی را ایجاد خواهیم کرد

```
from sklearn.datasets import make_regression, make_classification
X, y = make_regression()
from sklearn import dummy
dumdum = dummy.DummyRegressor()
dumdum.fit(X, y)
```

به طور پیش‌فرض، تخمین زننده فقط با گرفتن میانگین مقادیر و چندین بار نمایش آن‌ها، پیش‌بینی می‌کند

```
dumdum.predict(X)[:5]
output: array([17.10616876, 17.10616876, 17.10616876, 17.10616876,
17.10616876])
```

دو استراتژی دیگر نیز وجود دارد که می‌توانیم آن‌ها را امتحان کنیم. ما می‌توانیم یک ثابت عرضه شده را پیش‌بینی کنیم (constant=None). همچنین می‌توانیم مقدار میانه را پیش‌بینی کنیم. تأمین یک ثابت فقط در صورت ثابت بودن استراتژی در نظر گرفته خواهد شد.

```
predictors = [("mean", None),
("median", None),
("constant", 10)]
for strategy, constant in predictors:
 dumdum = dummy.DummyRegressor(strategy=strategy, constant=constant)
 dumdum.fit(X, y)
 print ("strategy: {}".format(strategy),
"; ".join(map(str,dumdum.predict(X)[:5])))
output:
```

```
strategy: mean
17.106168759399065,17.106168759399065,17.106168759399065,17.106168759399065,17.106168759399065
strategy: median
21.704256450120717,21.704256450120717,21.704256450120717,21.704256450120717,21.704256450120717
strategy: constant 10,10,10,10,10
```

در واقع ما چهار گزینه برای طبقه‌بندی داریم. این استراتژی‌ها مشابه حالت مداوم است و فقط به سمت مسائل طبقه‌بندی متمایل است

```
predictors = [("constant", 0),("stratified", None),("uniform", None),("most_frequent", None)]
#We'll also need to create some classification data:
X, y = make_classification()
for strategy, constant in predictors:
  dumdum = dummy.DummyClassifier(strategy=strategy, constant=constant)
  dumdum.fit(X, y)
  print( "strategy: {}".format(strategy),
",".join(map(str,dumdum.predict(X)[:5])))
output:
strategy: constant 0,0,0,0,0
strategy: stratified 1,0,0,1,0
strategy: uniform 0,1,0,1,0
strategy: most_frequent 0,0,0,0,0
```

مدل‌های خود را در برابر ساده‌ترین مدل‌ها آزمایش کنید، و این دقیقاً همان چیزی است که تخمین زننده‌ها به شما می‌دهند. به عنوان مثال، یک مدل ساختگی (غیر واقعی) را تصور کنید. در این مدل، تنها ۵٪ از مجموعه داده غیر واقعی است. بنابراین، ما فقط با حدس زدن که داده‌ها ساختگی نیستند، می‌توانیم یک مدل کاملاً مناسب را بدست آوریم.

با استفاده از دستور زیر این مدل را ایجاد می‌کنیم.

```
from sklearn.dummy import DummyClassifier
X, y = make_classification(20000, weights=[.95, .05])
dumdum = dummy.DummyClassifier(strategy='most_frequent')
dumdum.fit(X, y)
DummyClassifier(constant=None, random_state=None,
strategy='most_frequent')
from sklearn.metrics import accuracy_score
print( accuracy_score(y, dumdum.predict(X)))
output: 0.9453
```

انتخاب ویژگی

در این دستورالعمل، همراه با دو مورد دیگر، پیرامون انتخاب ویژگی‌ها خواهد بود. به همان روشی که برای یافتن یک پارامتر کلی مناسب از اعتبارسنجی متقابل استفاده کردیم، می‌توانیم یک زیر مجموعه عمومی مناسب از ویژگی‌ها پیدا کنیم. این شامل چندین روش مختلف است. ساده‌ترین ایده انتخاب یک متغیره است. روش‌های دیگر شامل کار با ترکیبی از ویژگی‌ها است. یکی دیگر از مزایای انتخاب ویژگی این است که می‌تواند بار جمع‌آوری اطلاعات را کاهش دهد. تصور کنید که شما یک مدل را روی زیرمجموعه بسیار کوچکی از داده‌ها ساخته‌اید. اگر همه چیز خوب پیش برود، ممکن است بخواهید برای پیش‌بینی مدل در کل زیر مجموعه داده‌ها، مقیاس‌بندی کنید. اگر چنین باشد، می‌توانید تلاش برای جمع‌آوری داده‌ها در آن مقیاس را کاهش دهید.

با انتخاب ویژگی تک متغیره، توابع امتیاز دهی مجددا استفاده می‌شوند و معیار قابل مقایسه‌ای را تعریف می‌کنند که ما می‌توانیم ویژگی‌ها را حذف کنیم. در این دستورالعمل، ما یک مدل رگرسیون را با حدود ۱۰۰۰۰ ویژگی را اعمال می‌کنیم، اما فقط ۱۰۰۰ امتیاز در نظر خواهیم گرفت. در اینجا از ویژگی‌های تک متغیره استفاده خواهیم کرد.

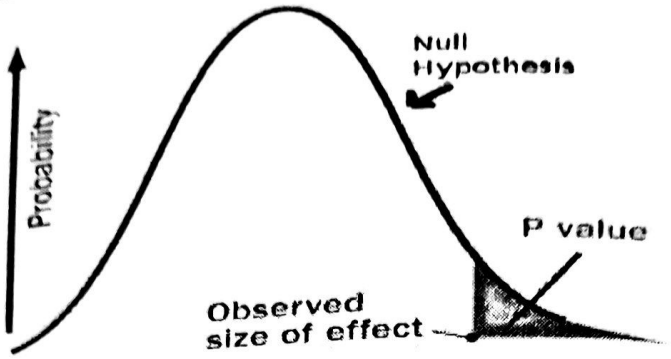
```
from sklearn import datasets
X, y = datasets.make_regression(1000, 10000)
```

اکنون که داده‌ها در دسترس ما است، ویژگی‌هایی را که با روش‌های مختلف گنجانده شده است مقایسه خواهیم کرد. در واقع زمانی که شما با تحلیل متن یا برخی از زمینه‌های بیوانفورماتیک سر و کار دارید این یک وضعیت بسیار معمول است.

ابتدا باید ماژول feature_selection را وارد کنیم

```
from sklearn import feature_selection
f, p = feature_selection.f_regression(X, y)
```

در اینجا، fscore مربوط به هر مدل خطی که فقط با یکی از ویژگی‌ها متناسب می‌شود. سپس می‌توانیم این ویژگی‌ها را مقایسه کنیم و بر اساس این مقایسه، ویژگی‌ها را حذف کنیم. p pvalue که با fvalue مرتبط است. در آمار، pvalue احتمال مقداری شدیدتر از مقدار فعلی آماره آزمون است. هرچه مقدار p کوچک‌تر باشد، شواهد قوی‌تری وجود دارد که شما باید فرضیه صفر را رد کنید. مقادیر P به صورت اعشاری بیان می‌شوند اگر آن‌ها را به درصد تبدیل کنید، درک اینکه چه مقدار هستند آسان‌تر است. به عنوان مثال، مقدار ۰/۰۲۵۴ p٪ برابر با ۲/۵۴ درصد است.



شکل (۷-۳): نمایش pvalue

```
f[:5]
output:array([0.92030738, 0.74768106, 1.81411307, 2.62102205, 1.10438456])
p[:5]
output:array([0.33762663, 0.3874192 , 0.17832138, 0.10577245, 0.29356028])
```

همانطور که می‌بینیم، بسیاری از pvalueها کاملاً بزرگ هستند. ما می‌خواهیم آن‌ها کاملاً کم باشند. بنابراین، می‌توانیم از NumPy برای یافتن مقادیر p کمتر از ۰/۰۵ استفاده کنیم. این‌ها ویژگی‌هایی هستند که ما برای تجزیه و تحلیل خود استفاده خواهیم کرد

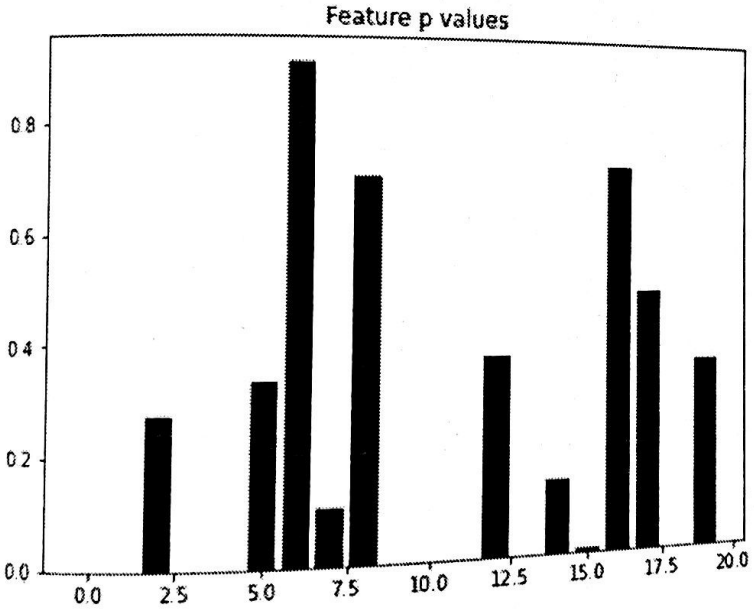
```
import numpy as np
idx = np.arange(0, X.shape[1])
features_to_keep = idx[p < .05]
len(features_to_keep)
output: 517
```

همانطور که می‌بینید، ما در واقع تعداد نسبتاً زیادی از ویژگی‌ها را حفظ می‌کنیم. بسته به کاربرد مدل، می‌توانیم pvalue را سخت‌تر کنیم. این باعث کاهش تعداد ویژگی‌های نگهداری شده می‌شود. گزینه دیگر استفاده از VarianceThreshold است. قبلا در مورد آن یاد گرفته ایم، اما مهم این است که توانایی ما در برازش (fit) کردن مدل‌ها تا حد زیادی بر اساس واریانس ایجاد شده توسط ویژگی‌ها است. اگر هیچ واریانسی وجود نداشته باشد، ویژگی‌های ما نمی‌توانند تغییر در متغیر وابسته را توصیف کنند. به علت عدم استفاده از متغیر نتیجه می‌توان آن را برای موارد بدون نظارت استفاده نمود. باید آستانه‌ای را تعیین کنیم که ویژگی‌ها را حذف می‌کند. برای انجام این کار، ما فقط میانه واریانس ویژگی را در نظر می‌گیریم و تأمین می‌کنیم:

```
var_threshold =feature_selection.VarianceThreshold(np.median(np.var(X,
axis=1)))
var_threshold.fit_transform(X).shape
output: (1000, 4861)
```

به طور کلی، همه این روش‌ها با برازش (fit) یک مدل پایه با یک ویژگی واحد کار می‌کنند. بسته به اینکه مسئله کلاسه‌بندی داشته باشیم یا رگرسیون، می‌توانیم از تابع امتیازدهی مناسب استفاده کنیم. بیایید به یک مسئله کوچک‌تر نگاه کنیم و ببینیم که انتخاب ویژگی چگونه برخی از ویژگی‌ها را از بین می‌برد. ما از همان تابع امتیازدهی در مثال اول استفاده خواهیم کرد.

```
X, y = datasets.make_regression(10000, 20)
f, p = feature_selection.f_regression(X, y)
%matplotlib inline
from matplotlib import pyplot as plt
f, ax = plt.subplots(figsize=(7, 5))
ax.bar(np.arange(20), p, color='k')
ax.set_title("Feature p values")
```



شکل (۷-۴): نمودار pvalue داده‌ها

همانطور که می‌بینیم، بسیاری از ویژگی‌ها حفظ نمی‌شوند.

انتخاب ویژگی در نرم L1

در اینجا با برخی ایده‌های مشابه آنچه که در دستورالعمل رگرسیون LASSO دیده‌ایم کار می‌کنیم. اکنون ما قصد داریم در این مرحله یک گام جلوتر برداریم و از پراکندگی مرتبط با هنجارهای L1 برای پیش پردازش ویژگی‌ها استفاده کنیم.

ما از مجموعه داده‌های دیابت استفاده خواهیم کرد که با رگرسیون برازش می‌شوند. ابتدا یک مدل رگرسیون خطی پایه با اعتبارسنجی متقاطع ShuffleSplit متناسب یا برازش خواهیم کرد. پس از انجام این کار، از رگرسیون LASSO برای یافتن ضرایب صفر،که از پنالتی L1 بهره می‌برند، استفاده خواهیم کرد. این به ما کمک می‌کند تا از بیش برازش جلوگیری کنیم. به عبارت دیگر، اگر بیش برازش رخ دهد، داده‌ها به خوبی تعمیم داده نمی‌شوند.

ما قصد داریم مراحل زیر را انجام دهیم:

- مجموعه داده را بارگیری کنید.
- یک مدل رگرسیون خطی پایه برازش کنید.
- از انتخاب ویژگی برای حذف ویژگی‌های غیر ضروری استفاده کنید.
- رگرسیون خطی را دوباره برازش کرده و بررسی کنید که در مقایسه با مدل کاملاً برازش شده چقدر مناسب است.

```
import sklearn.datasets as ds
diabetes = ds.load_diabetes()
X = diabetes.data
y = diabetes.target
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
```

mean_squared_error و make_scorer را وارد کنیم. از اعتبارسنجی مدل_selection، طرح اعتبارسنجی ShuffleSplitcross و امتیاز دهنده اعتبارسنجی cross_val_scorecross را وارد کنید. پیش بروید و باmean_squared_errormetric به تابع امتیاز دهید

```
from sklearn.metrics import make_scorer, mean_squared_error
from sklearn.model_selection import cross_val_score,ShuffleSplit
shuff = ShuffleSplit(n_splits=10, test_size=0.25, random_state=0)
score_before =
cross_val_score(lr,X,y,cv=shuff,scoring=make_scorer(mean_squared_error,gre
ater_is_better=False)).mean()
score_before
output: -3053.393446308266
```

بنابراین اکنون که برازش دادیم، بیایید پس از حذف هر ویژگی با ضریب صفر، آن را بررسی کنیم. رگرسیون LASSO را متناسب کنیم

```
from sklearn.linear_model import LassoCV
lasso_cv = LassoCV()
lasso_cv.fit(X,y)
lasso_cv.coef_
```

```
output: array([ -6.49684455, -235.99640534, 521.73854261, 321.066892[illegible],
-569.4426838 , 302.45627915,  -0.   , 143.6995665 ,
669.92633112,  66.83430445])
```

اولین ویژگی را حذف خواهیم کرد.

```
import numpy as np
columns = np.arange(X.shape[1])[lasso_cv.coef_ != 0]
columns
output: array([0, 1, 2, 3, 4, 5, 7, 8, 9])
```

حال مدل را با ویژگی‌های خاص برازش می‌کنیم

```
score_afterwards = cross_val_score(lr,X[:,columns],y,cv=shuff,
scoring=make_scorer(mean_squared_error,greater_is_better=False)).mean()
score_afterwards
output: -3025.7583947461308
```

امتیاز آن خیلی بهتر از نمره قبل نیست، حتی اگر یک ویژگی غیر اطلاعاتی را حذف کنیم.

در مثالی دیگر ابتدا، ما می‌خواهیم یک مجموعه داده رگرسیون با بسیاری از ویژگی‌های غیر اطلاعاتی ایجاد کنیم

```
X, y = ds.make_regression(noise=5)
```

یک نمونه ShuffleSplit با ۱۰ تکرار، n_splits = 10 ایجاد کنید. نمره اعتبار متقابل رگرسیون خطی ساده را اندازه‌گیری کنید

```
shuff = ShuffleSplit(n_splits=10, test_size=0.25, random_state=0)
score_before =
cross_val_score(lr,X,y,cv=shuff,scoring=make_scorer(mean_squared_error,gre
ater_is_better=False)).mean()
```

با استفاده از LassoCV می‌توان ستون‌های غیر ضروری را حذف نمود

```
lasso_cv = LassoCV()
lasso_cv.fit(X,y)
```

به امتیاز نهایی نگاه کنید

```
columns = np.arange(X.shape[1])[lasso_cv.coef_ != 0]
score_afterwards = cross_val_score(lr,X[:,columns],y,cv=shuff,

scoring=make_scorer(mean_squared_error,greater_is_better=False)).mean()
print ("Score before:",score_before)
print ("Score after: ",score_afterwards)
output: Score before: -7568.407123417002
Score after: -26.130133311272942
```

پس از حذف ویژگی‌های غیر مفید، امتیاز برازش بهتر شده است

مدل‌های ماندگار با joblib یا pickle

در این دستورالعمل، ما قصد داریم نشان دهیم که چگونه می‌توانید مدل را مجددا ذخیره نمود. به عنوان مثال، شما ممکن است بخواهید در واقع از یک مدل برای پیش‌بینی سریعه و تصمیم‌گیری خودکار استفاده کنید.

```
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier
X, y = make_classification()
dt = DecisionTreeClassifier()
dt.fit(X, y)
```

یک راه ذخیره مدل با joblib می‌باشد

```
import joblib
joblib.dump(dt, "dtree.clf")
```

بارگذاری مدل را با joblib انجام می‌دهیم

```
pulled_model = joblib.load("dtree.clf")
y_pred = pulled_model.predict(X)
```

ذخیره مدل با pickle نیز امکان‌پذیر است مانند

```
import pickle
f = open("dtree.save", 'wb')
pickle.dump(dt,f, protocol = pickle.HIGHEST_PROTOCOL)
f.close()
f = open("dtree.save", 'rb')
```

بارگذاری مدل با pickle را انجام می‌دهیم

```
return_tree = pickle.load(f)
f.close()
return_tree
```